

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 4

Comparative Study of Deep Convolutional Neural Network Architectures Using Transfer Learning

Degree & Branch			B.Tech Artificial Intelligence & Data Science	Semester V
Subject Name	Code	&	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

The objectives of this experiment are:

- To study the evolution of deep Convolutional Neural Network architectures.
- To compare important CNN architectures including LeNet-5, AlexNet, VGG16, GoogleNet and ResNet.
- To understand the concept of transfer learning.
- To implement transfer learning using a pretrained CNN.
- To fine tune selected layers of a pretrained CNN.
- To evaluate the performance of the model using standard classification metrics.

2. Learning Outcomes

After completing this experiment, the student will be able to:

1. Explain the evolution of modern CNN architectures.
2. Differentiate between LeNet-5, AlexNet, VGG16, GoogleNet and ResNet.
3. Understand the principles of transfer learning.
4. Implement transfer learning using a pretrained CNN.
5. Fine tune selected pretrained convolutional layers.
6. Evaluate image classification models using Accuracy, Precision, Recall and F1-score.

3. Introduction

Deep Convolutional Neural Networks (CNNs) have become fundamental models for image classification and computer vision tasks. CNNs learn hierarchical representations of images through convolutional filters, activation functions and pooling operations.

The development of CNN architectures has progressed from relatively small networks such as LeNet-5 to deeper and more sophisticated architectures such as AlexNet, VGG16, GoogleNet and ResNet. These architectures introduced important concepts such as deeper convolutional networks, parallel convolutional branches and residual connections.

Training deep neural networks from scratch can require large datasets, considerable computational resources and long training times. Transfer learning addresses this problem by reusing knowledge learned by a model on a large dataset.

In this experiment, MobileNetV2 pretrained on ImageNet is used for classification of CIFAR-10 images. The pretrained convolutional base is initially frozen and a new classification head is trained. Selected layers are subsequently unfrozen and fine tuned using a smaller learning rate.

4. Evolution of CNN Architectures

The evolution of CNN architectures has focused on increasing feature learning capability while improving computational efficiency and optimization.

- **LeNet-5:** One of the earliest successful CNN architectures, mainly designed for handwritten digit recognition.
- **AlexNet:** Demonstrated the effectiveness of deep CNNs on large-scale image classification and popularized the use of ReLU activations and GPU-based training.
- **VGG16:** Used a deeper architecture based mainly on small 3×3 convolutional filters.
- **GoogLeNet:** Introduced the Inception module, allowing multiple convolutional operations with different receptive fields to operate in parallel.
- **ResNet:** Introduced residual connections to make training very deep neural networks easier.

5. LeNet-5

LeNet-5 is an early convolutional neural network architecture developed for handwritten digit recognition.

Its architecture consists of convolutional and pooling layers followed by fully connected layers.

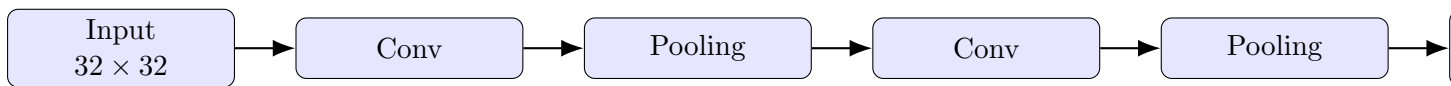


Figure 1: Simplified LeNet-5 Architecture

LeNet demonstrated that convolutional operations could automatically learn useful spatial features from images and became a foundation for later CNN architectures.

6. AlexNet

AlexNet was a major milestone in deep learning for image classification. It achieved strong performance on the ImageNet Large Scale Visual Recognition Challenge.

Important features of AlexNet include:

- ReLU activation functions.
- GPU-based training.
- Multiple convolutional layers.
- Dropout for reducing overfitting.
- Data augmentation.

The use of ReLU significantly improved optimization compared with traditional sigmoid or tanh activations.

7. VGG16

VGG16 is a deep CNN architecture containing 16 weight layers. It uses small 3×3 convolutional filters throughout most of the network.

The architecture can be summarized as:

Convolution $\rightarrow$ ReLU $\rightarrow$ Pooling $\rightarrow$ Fully Connected

The repeated use of small convolutional filters allows VGG16 to learn hierarchical visual representations while maintaining a relatively simple architecture.

8. GoogleNet and ResNet

8.1 GoogleNet

GoogleNet introduced the Inception architecture. Instead of relying on a single convolutional filter size, the Inception module applies multiple operations in parallel.

- 1×1 convolution
- 3×3 convolution
- 5×5 convolution
- Pooling

The outputs are concatenated to form a richer feature representation.

8.2 ResNet

ResNet introduced residual learning using shortcut connections.

A residual block can be represented as:

$$y = F(x) + x$$

where $F(x)$ represents the learned residual mapping and x is the identity shortcut.

Residual connections make it easier to train very deep networks and help reduce the degradation problem.

9. Comparison of CNN Architectures

Architecture	Main Contribution	Characteristic	Limitation
LeNet-5	Early CNN architecture	Small convolutional network	Limited capacity for complex datasets
AlexNet	Deep CNN and ReLU	GPU training and dropout	Large computational requirements
VGG16	Deep network with 3×3 filters	Simple and uniform architecture	Large number of parameters
GoogleNet	Inception modules	Parallel multi-scale feature extraction	More complex architecture
ResNet	Residual connections	Very deep networks	Higher computational complexity

Table 1: Comparison of Major CNN Architectures

10. Transfer Learning

Transfer learning is a machine learning technique in which knowledge learned from one task is reused for another related task.

In computer vision, a CNN pretrained on a large dataset such as ImageNet learns general visual features including:

- Edges
- Textures
- Shapes
- Patterns
- Higher-level visual structures

These learned features can be transferred to a new dataset.

The general transfer learning process used in this experiment is:

1. Load a pretrained MobileNetV2 model.
2. Remove the original classification head.
3. Freeze the pretrained convolutional base.
4. Add a new classification head for CIFAR-10.
5. Train the new classification head.
6. Unfreeze selected final layers.
7. Fine tune using a smaller learning rate.

11. Dataset

The CIFAR-10 dataset was used for this experiment.

CIFAR-10 contains 60,000 colour images belonging to 10 classes.

Property	Value
Training Images	50,000
Testing Images	10,000
Number of Classes	10
Original Image Size	$32 \times 32 \times 3$
Image Type	RGB

Table 2: CIFAR-10 Dataset Details

The ten classes are:

Airplane, Automobile, Bird, Cat, Deer, Dog, Frog, Horse, Ship, Truck

12. Dataset Preparation and Preprocessing

The original CIFAR-10 images have a spatial resolution of 32×32 pixels. Since MobileNetV2 was designed for larger input images, the images were resized to 96×96 pixels.

The preprocessing steps were:

1. Load the CIFAR-10 dataset.
2. Convert image values to floating-point representation.
3. Normalize the image values.
4. Resize the images from 32×32 to 96×96 .
5. Apply MobileNetV2 preprocessing.
6. Create TensorFlow dataset pipelines.
7. Shuffle the training data.
8. Use a batch size of 32.

The input shape to the final model is therefore:

$$96 \times 96 \times 3$$

13. MobileNetV2 Architecture

MobileNetV2 is a lightweight CNN architecture designed for efficient image classification and deployment on resource-constrained devices.

It uses depthwise separable convolutions and inverted residual blocks to reduce computational cost.

The pretrained MobileNetV2 model used in this experiment was obtained with ImageNet weights.

The original classification head was removed using:

include_top=False

The convolutional base was then connected to a new classification head.
The resulting architecture is:

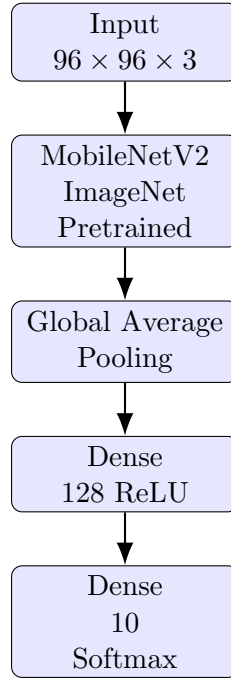


Figure 2: MobileNetV2 Transfer Learning Architecture

14. Model Configuration

The transfer learning model was configured as follows:

Parameter	Value
Base Model	MobileNetV2
Pretrained Weights	ImageNet
Input Size	$96 \times 96 \times 3$
Classification Classes	10
Pooling	Global Average Pooling
Dense Layer	128 neurons
Activation	ReLU
Output Layer	10 neurons
Output Activation	Softmax
Optimizer	Adam
Initial Learning Rate	0.001
Batch Size	32
Initial Training Epochs	10
Fine-Tuning Learning Rate	0.0001
Fine-Tuning Epochs	5

Table 3: Model Configuration

15. Model Training

During the initial transfer learning phase, the MobileNetV2 convolutional base was frozen.

Only the newly added classification layers were trained.

The model was compiled using the Adam optimizer with a learning rate of 0.001.

The loss function used was sparse categorical cross-entropy:

$$L = -\frac{1}{N} \sum_{i=1}^N \log p(y_i|x_i)$$

The model was trained for 10 epochs using a batch size of 32.

The training process was monitored using training and validation accuracy and loss.

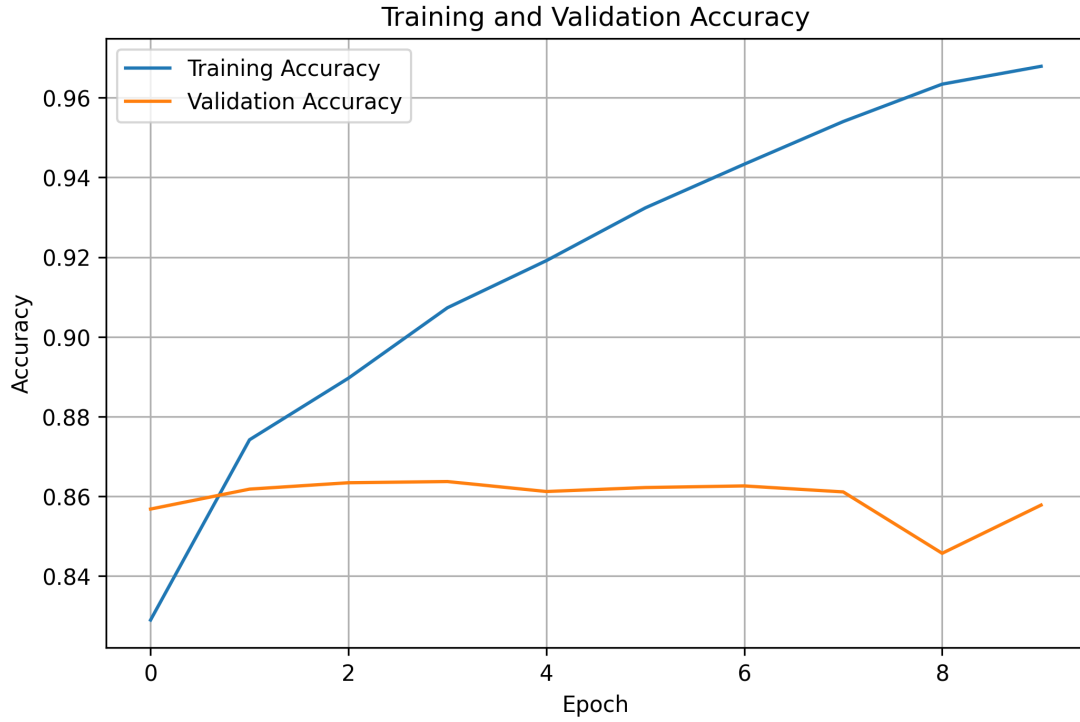


Figure 3: Training and Validation Accuracy

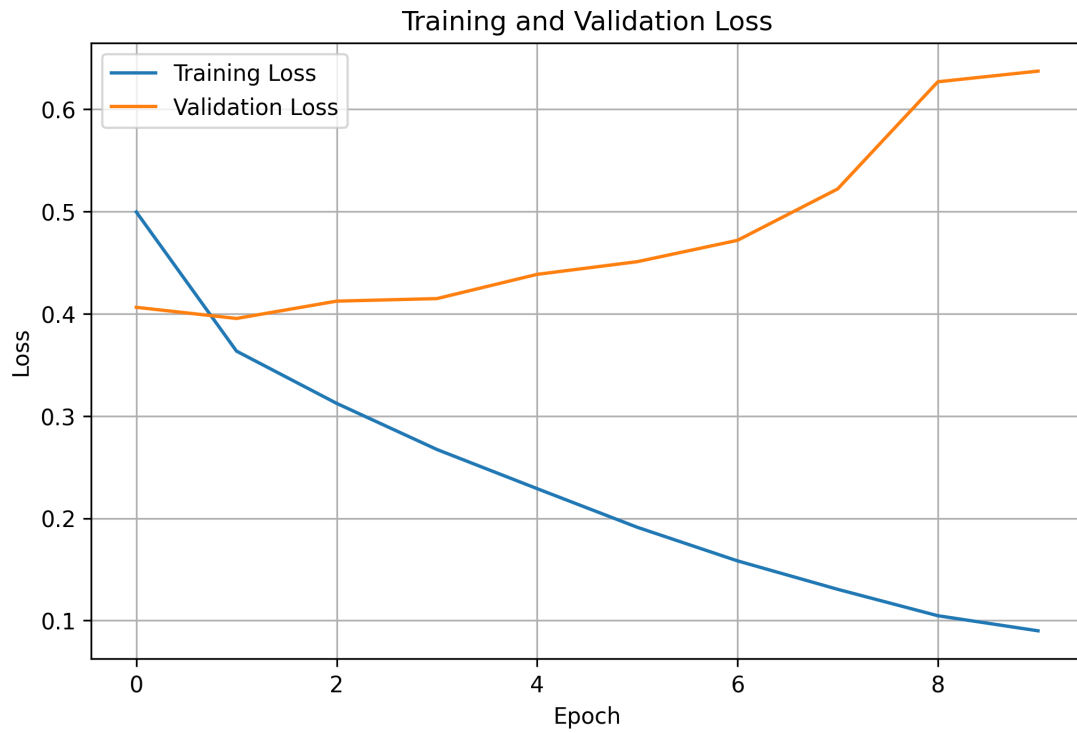


Figure 4: Training and Validation Loss

16. Fine Tuning

After the initial transfer learning stage, selected final layers of the MobileNetV2 convolutional base were unfrozen.

The final 20 layers of the base model were made trainable while the earlier layers remained frozen.

A smaller learning rate of 0.0001 was used during fine tuning to prevent large changes to the pretrained features.

The model was trained for an additional 5 epochs.

The purpose of fine tuning was to allow the higher-level pretrained features to adapt to the CIFAR-10 dataset.

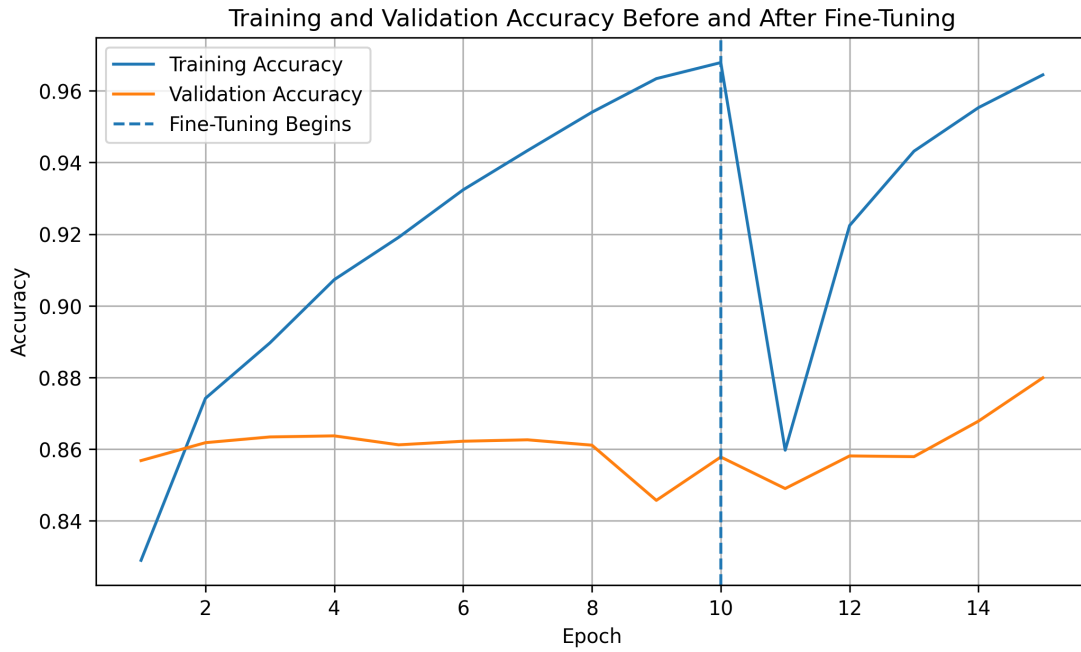


Figure 5: Fine-Tuning Training and Validation Accuracy

17. Model Evaluation

The final fine-tuned model was evaluated using the CIFAR-10 test set.

The following metrics were calculated:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

Accuracy is defined as:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Precision is defined as:

$$Precision = \frac{TP}{TP + FP}$$

Recall is defined as:

$$Recall = \frac{TP}{TP + FN}$$

The F1-score is the harmonic mean of precision and recall:

$$F1 = 2 \frac{Precision \cdot Recall}{Precision + Recall}$$

18. Results

18.1 Transfer Learning and Fine Tuning Results

The model achieved an accuracy of 85.78% before fine tuning.

After fine tuning, the accuracy increased to 87.99%.

The improvement was:

$$87.99 - 85.78 = 2.21$$

percentage points.

Metric	Result
Before Fine Tuning Accuracy	85.78%
After Fine Tuning Accuracy	87.99%
Accuracy Improvement	2.21 percentage points
Precision	87.99%
Recall	87.99%
F1-score	87.94%
Total Parameters	2,423,242
Initial Training Time	243.89 seconds
Fine Tuning Time	186.98 seconds
Total Training Time	430.87 seconds

Table 4: Final Experimental Results

18.2 CNN Architecture Comparison

The major CNN architectures studied in this experiment are compared below.

Model	Key Idea	Architecture	Main Advantage
LeNet-5	Convolution and pooling	Shallow CNN	Simple image recognition
AlexNet	Deep CNN and ReLU	Multiple convolutional layers	High classification performance
VGG16	Small 3×3 filters	Very deep sequential CNN	Simple and uniform architecture
GoogLeNet	Inception modules	Parallel convolution paths	Multi-scale feature extraction
ResNet	Residual learning	Shortcut connections	Efficient training of deep networks
MobileNetV2	Depthwise separable convolution	Lightweight CNN	Efficient transfer learning

Table 5: Comparison of CNN Architectures

19. Confusion Matrix and Misclassification Analysis

The confusion matrix provides a class-wise representation of the predictions made by the final fine-tuned model.

Diagonal elements represent correctly classified samples, while off-diagonal elements represent misclassified samples.

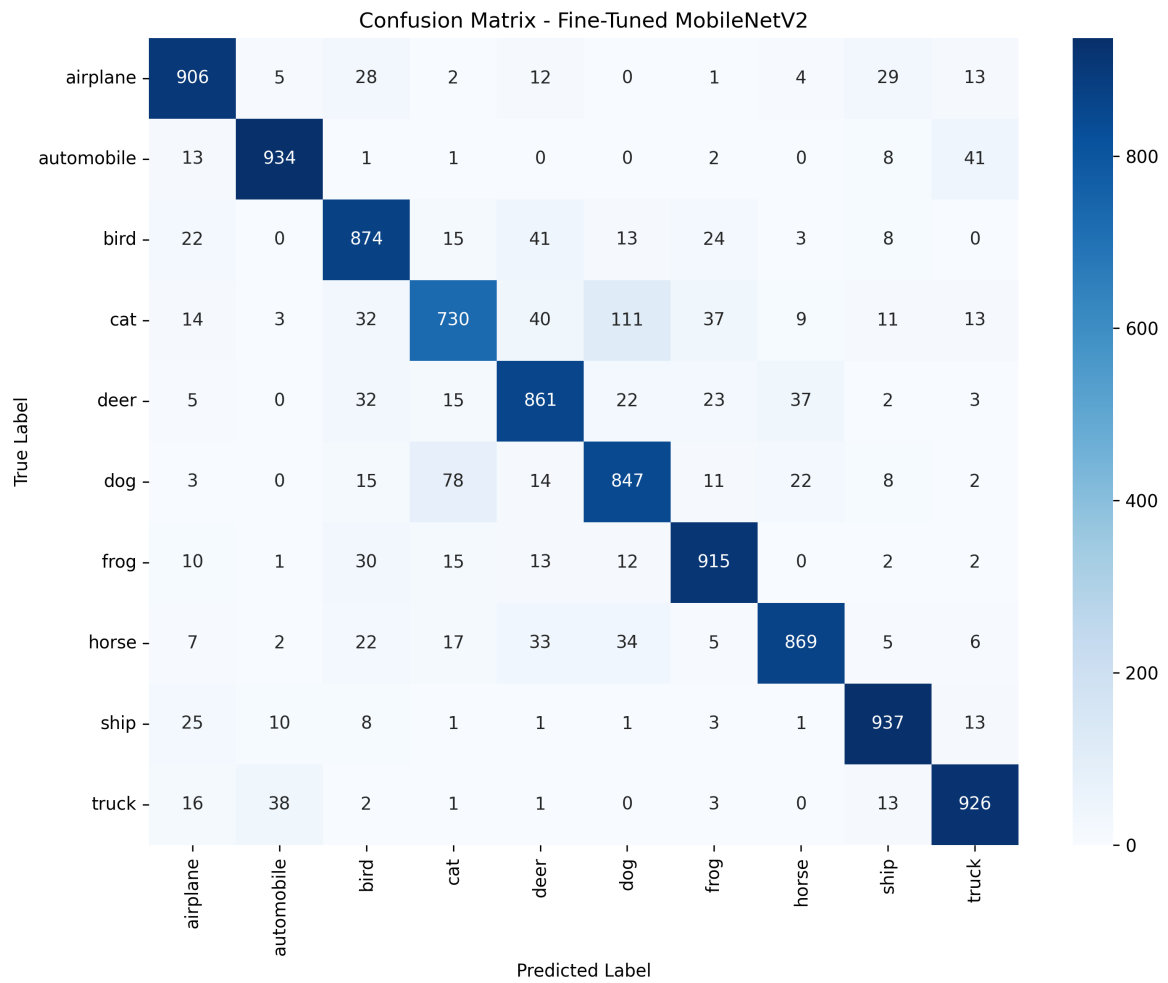


Figure 6: Confusion Matrix of Fine-Tuned MobileNetV2

The confusion matrix can be used to identify classes that are more difficult to distinguish. CIFAR-10 contains visually similar classes, such as cat and dog, which can result in increased confusion between those classes.

19.1 Misclassified Images

A selection of misclassified images was also visualized to understand the errors made by the model.

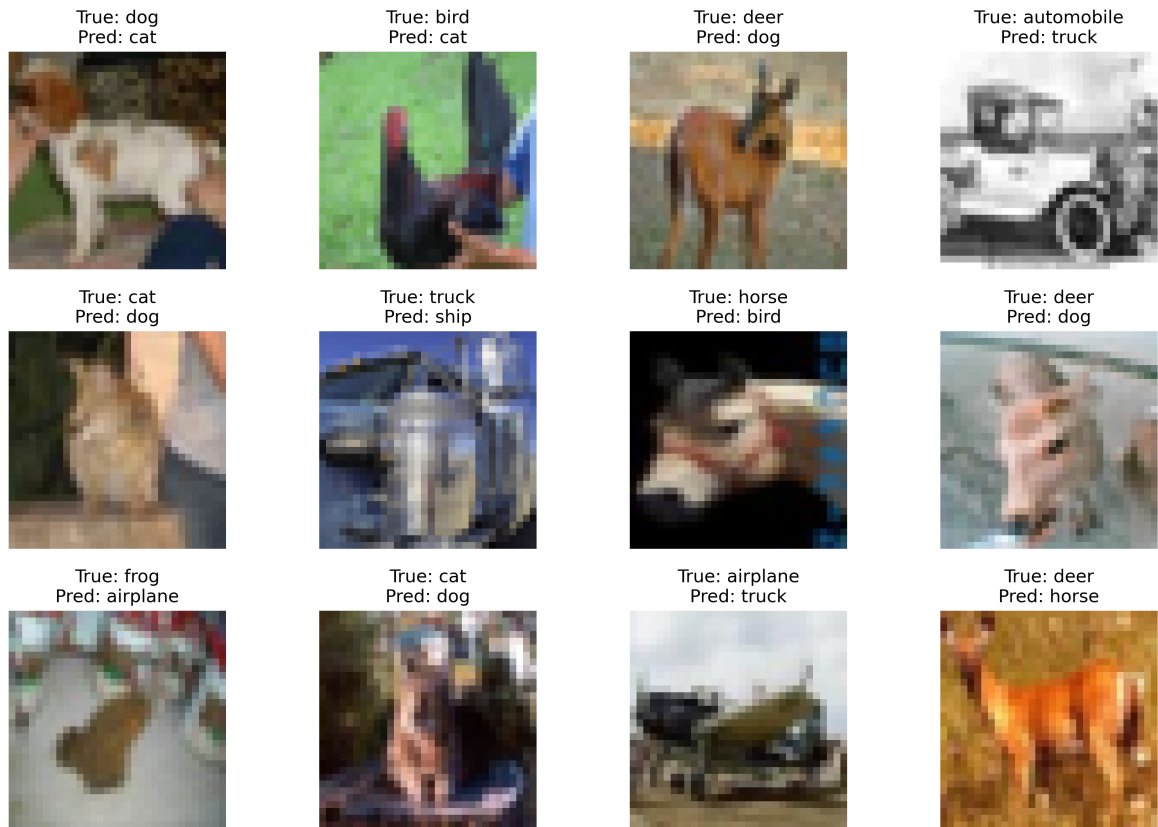


Figure 7: Sample Misclassified Images

Misclassification analysis provides additional insight into cases where the visual features of different classes overlap.

20. Discussion

The experiment demonstrates the usefulness of transfer learning for image classification.

MobileNetV2 pretrained on ImageNet achieved an accuracy of 85.78% before fine tuning. The pretrained convolutional base provided useful visual representations that could be transferred to the CIFAR-10 classification task.

After unfreezing the final portion of the network and fine tuning it using a learning rate of 0.0001, the accuracy increased to 87.99%.

The improvement of 2.21 percentage points indicates that adapting the higher-level pretrained features to the target dataset was beneficial.

The final model achieved a weighted precision of 87.99%, weighted recall of 87.99%, and weighted F1-score of 87.94%.

The total number of parameters in the model was 2,423,242.

The initial training phase required 243.89 seconds, while fine tuning required an additional 186.98 seconds. The total training time was 430.87 seconds.

21. Conclusion

In this experiment, the evolution of important CNN architectures was studied, including LeNet-5, AlexNet, VGG16, GoogleNet and ResNet.

Transfer learning was implemented using MobileNetV2 pretrained on ImageNet. The CIFAR-10 dataset was resized and preprocessed before being passed through the pretrained network.

The initial transfer learning model achieved an accuracy of 85.78%. After fine tuning the final portion of the MobileNetV2 network, the accuracy increased to 87.99%.

Therefore, fine tuning resulted in an improvement of 2.21 percentage points.

The experiment demonstrates that pretrained CNN models can provide effective feature representations for new image classification tasks, while fine tuning allows those representations to be adapted to the target dataset.

Source Code

The complete implementation, including dataset preparation, transfer learning, model training, fine tuning, evaluation and visualization, is available in the GitHub repository.

GitHub Repository:

<https://github.com/KPR-24110081/Deep-Learning-Lab/tree/main/Ex-4>